ADHD Focus Coach

API Documentation · Corrected API Contract · Version 1.1

Base path: /api/v1 **Authentication:** Clerk bearer token on protected endpoints
Architecture rule: Mobile communicates only with FastAPI. FastAPI performs authentication/authorization, business logic, validation and persistence. PostgreSQL is the source of truth. AI operations run through the AIProvider abstraction.

Access rule: Clerk authenticates the caller; FastAPI authorizes using application role and resource ownership. Children access only their own child-scoped resources and child-facing execution operations. Caregivers access resources belonging to their own children. Unauthorized combinations return 403.

1. API Endpoint Reference

Topic	Method	API	Who	Purpose / Contract
Authentication	**GET**	**/auth/me**	**Both**	Return the authenticated application user and role resolved from Clerk identity.
Children	**POST**	**/children**	**Caregiver**	Create a child profile. Ownership comes from the authenticated caregiver; caregiver_id is never supplied by the client.
Children	**GET**	**/children**	**Caregiver**	List children belonging to the authenticated caregiver.
Children	**GET**	**/children/{child_id}**	**Both**	Retrieve the profile only when the caller is the owning caregiver or the child mapped to that profile.
Children	**PATCH**	**/children/{child_id}**	**Caregiver**	Update the child profile; caregiver ownership is checked server-side.
Children	**DELETE**	**/children/{child_id}**	**Caregiver**	Delete the child and defined child-owned data according to the application deletion policy.
Children	**GET**	**/children/{child_id}/export**	**Caregiver**	Export the child's application data in a structured response.
Tasks	**POST**	**/tasks**	**Caregiver**	Create a task. If scheduled_at exists, create/update the associated PENDING notification in the same business operation.
Tasks	**GET**	**/tasks**	**Both**	List tasks accessible to the caller: caregiver sees their children's tasks; child sees only their own tasks.
Tasks	**GET**	**/tasks/{task_id}**	**Both**	Return task resource details only. Authoritative focus execution state is returned by /focus-state.
Tasks	**GET**	**/tasks/{task_id}/steps**	**Both**	Return the task's ordered TaskSteps by position after authorization.
Tasks	**PATCH**	**/tasks/{task_id}**	**Caregiver**	Update task fields. Schedule changes update/cancel the related pending notification.
Tasks	**DELETE**	**/tasks/{task_id}**	**Caregiver**	Delete/cancel the task and cancel any related pending notification.
AI — Breakdown	**POST**	**/tasks/{task_id}/breakdown**	**Caregiver**	Generate the initial 3–5 child-appropriate steps, each below 5 minutes. If steps already exist, return 409 and preserve the existing breakdown.
AI — Refine	**POST**	**/tasks/{task_id}/steps/{step_id}/refine**	**Internal***	Generate a clearer/easier version of the current step. Invoked by the child-facing stuck flow; not an unrestricted child action.
AI — Decompose	**POST**	**/tasks/{task_id}/steps/{step_id}/decompose**	**Internal***	Split a difficult/oversized step into smaller executable steps. Invoked by the child-facing stuck flow; not an unrestricted child action.
Focus	**POST**	**/tasks/{task_id}/start**	**Child**	Start focus execution and initialize the deterministic FSM/runtime state.
Focus	**GET**	**/tasks/{task_id}/focus-state**	**Child**	Return authoritative focus state, active step position and break/inactivity information. Read-only; never updates last_activity_at.
Focus	**POST**	**/tasks/{task_id}/steps/{step_id}/complete**	**Child**	Complete the current step using expected_step_position. Return NEXT_STEP or TASK_COMPLETED. Position mismatch returns 409.
Focus	**POST**	**/tasks/{task_id}/steps/{step_id}/stuck**	**Child**	Record the stuck reason and route it to REFINE, DECOMPOSE or BREAK. Maximum 3 AI adaptations per step; after the cap, use deterministic break/fallback.
Focus	**POST**	**/tasks/{task_id}/break**	**Child**	Move active focus into BREAK, preserve the current step, and start the configured deterministic break duration.
Focus	**POST**	**/tasks/{task_id}/resume**	**Child**	Resume the preserved current step, reset last_activity_at and inactivity monitoring.
Feedback	**POST**	**/tasks/{task_id}/feedback**	**Child**	Submit the short post-task FAQ for that task. Feedback is tied to the task for authorization and duplicate-control.
Feedback	**GET**	**/children/{child_id}/feedback**	**Caregiver**	Read structured feedback for an owned child.
Progress	**GET**	**/progress**	**Both**	Return accessible progress data using stored/precomputed values rather than heavy live aggregation.
Progress	**GET**	**/progress/summary**	**Both**	Return stored completion and recent-progress summary for the caller's authorized scope.
Progress	**GET**	**/progress/streak**	**Both**	Return the stored current streak. Streak computation is performed by the scheduled progress job.
Notifications	**GET**	**/notifications**	**Both**	List notifications accessible to the authenticated user.

Topic	Method	API	Who	Purpose / Contract
Devices	**POST**	**/devices**	**Both**	Register the caller's Expo push token and device platform. User identity comes from Clerk.
Devices	**DELETE**	**/devices/{device_id}**	**Both**	Deactivate/remove the caller's registered push device, including on logout.
Telemetry	**POST**	**/telemetry/events**	**Child**	Submit a batch of meaningful events. Each event has a unique event_id; duplicates are ignored. Return quickly so telemetry does not block the child UI.

* **Internal** means the operation is invoked by the child-facing /stuck flow; it is not an unrestricted client action. This keeps the adaptation cap and deterministic orchestration centralized.

2. Request / Response and Safety Rules

Authorization: Every protected request is authenticated through Clerk. FastAPI resolves the application User and role, then checks child/resource ownership. Client-supplied caregiver_id is never trusted.

State-changing focus requests: Completion and other execution mutations carry expected_step_position. The server compares it with the authoritative current step; mismatch returns 409 Conflict. This protects against double taps and concurrent devices.

AI idempotency: AI mutation requests use the Idempotency-Key header. Repeating the same key within the retention window replays the stored result instead of issuing another LLM request.

AI rate/cost control: The child-facing /stuck flow permits at most 3 AI adaptations for a given step. After the cap, no further LLM call is made; the flow uses the deterministic break/fallback path.

Breakdown repeat: Initial breakdown is allowed only when the task has no existing TaskSteps. If steps already exist, return 409 Conflict and leave the current breakdown unchanged.

AI validation: AI output must pass schema/Pydantic validation, business rules (3–5 steps for breakdown; each estimated duration below 5 minutes), and safety validation before persistence/display. On validation failure, retry once. If the retry is still invalid, preserve existing state and use the defined deterministic/plain fallback.

Stuck response: The response explicitly reports REFINE, DECOMPOSE or BREAK. The LLM never controls FSM state, queue ordering or persistence.

Complete-step response: The response explicitly reports NEXT_STEP when another step becomes current, or TASK_COMPLETED when no steps remain.

Focus-state polling: While a task is actively displayed, the mobile client polls /focus-state at a documented 5-second interval. The endpoint is read-only and never changes last_activity_at. Only meaningful actions such as DONE, I'M STUCK, LET'S GO, TAKE A BREAK and RESUME reset inactivity monitoring.

Task scheduling & notifications: Creating a scheduled task creates its PENDING notification. Changing scheduled_at updates the related pending notification; removing the schedule cancels it; deleting the task cancels its pending notification. There is no public POST /notifications.

Telemetry: Telemetry is submitted in batches. Each event has a unique event_id so retries are deduplicated. The endpoint returns quickly so telemetry does not delay the child's screen; telemetry is not authoritative task state.

Progress: Progress summary/streak values are computed by the scheduled progress job and stored. GET /progress, /summary and /streak read stored values rather than recalculating heavy aggregates on every request.

Worker indexes: The database provides indexes for worker queries, including active-task inactivity lookup using focus state/status with last_activity_at as applicable, and pending-notification lookup using status with scheduled_at.

Push devices: The mobile app registers its Expo push token through POST /devices. One user may have multiple devices. Logout removes/deactivates the device registration. NotificationService uses stored tokens when calling the Expo push provider.

Focus state authority: GET /tasks/{task_id} is the task-resource endpoint. GET /tasks/{task_id}/focus-state is the single authoritative endpoint for live execution state. The two endpoints do not compete as sources of truth.

Persistence boundary: There is no persistent FocusSession, StepExecution or SessionStep entity. Current execution state is maintained with Task fields such as current_step_position, focus_state and last_activity_at, while the runtime Deque remains an application data structure.

3. Common API Response Codes

Code	Meaning
200	Successful read, update or action.
201	Resource created successfully.
204	Successful deletion with no response body.
400	Invalid request or business-level request error.

Code	Meaning
401	Missing or invalid authentication credentials.
403	Authenticated user is not authorized for the resource/action.
404	Requested resource was not found.
409	State/resource conflict, including step-position mismatch or duplicate initial breakdown.
422	Request failed schema/validation checks.
429	Rate limit exceeded.
500 / 503	Internal server error or unavailable external dependency.

4. Implementation Notes for Route Contracts

• Request bodies contain only client-owned inputs. Identity, caregiver ownership and authorized child scope are derived from Clerk and server-side lookup.

• Protected role/ownership permissions are part of the API contract, not only UI restrictions.

• Redis may support AI-response caching, rate limiting and idempotency-result replay, but PostgreSQL remains authoritative for tasks, steps, notifications, feedback, context and telemetry.

• Notification delivery is an internal provider integration. The public API registers devices and reads notifications; it does not expose the push provider directly to the mobile app.

• The API remains a modular-monolith contract; no Kafka, RabbitMQ, separate notification service or separate AI microservice is required.